Lab 3 and Lab 5 - interview study guide

Use this as a reference for questions, not as a speech. For each experiment, understand **what you changed, why, what happened, and what you can conclude**. You do not need to memorize every number.

Concepts that apply to both labs

- **Training / validation / test:** training updates weights; validation guides model choices; a separate test set evaluates the chosen model. The food experiments report validation accuracy, not independent test accuracy.
- **Loss / accuracy:** cross-entropy penalizes low probability for the correct answer; accuracy only counts correct predictions. Loss can worsen even when accuracy stays similar, because confidence changes.
- **Overfitting:** training improves while validation stalls or worsens. A small gap alone is not enough: both results could be poor.
- **Backpropagation / optimizer:** backpropagation computes gradients using the chain rule; SGD or Adam uses them to update weights. The learning rate controls update size.
- **Parameters / hyperparameters:** weights are learned; settings such as learning rate, layer width and dropout are chosen. Changing several settings together makes it hard to identify which caused an improvement.

Lab 3 - `1_Simple2NN.ipynb`: fully connected network

What you did: implemented a NumPy network with one hidden layer: input -> linear -> ReLU -> linear class scores. You computed cross-entropy plus L2 regularization, gradients and mini-batch SGD updates. CIFAR-10 images became 3,072 input values.

Result: hyperparameter tuning improved validation accuracy from **28.6% to 52.5%**; the selected model achieved **49.96% test accuracy**. Its hidden size was 100, learning rate 0.001 and L2 strength 0.25. The toy example checked that training could reduce loss.

- **Why ReLU?** It adds nonlinearity. Without nonlinear activations, several linear layers still form a linear mapping.
- **Why L2?** It penalizes large weights and can reduce overfitting; too much can make learning useful patterns harder.
- **Why tune using validation data?** Selecting settings using the test set would make the final test result less independent.

Related slides: 3.3, PDF pp. 24, 28, 30-32; 3.4, pp. 7, 20.

Lab 3 - `2_SimpleCNN.ipynb`: CNN and regularization

What you did: trained three convolution/pooling blocks and a dense classifier on five food categories: 4,000 training and 1,000 validation images. Compared no dropout, dropout, augmentation, and a larger model.

Version	Final validation accuracy	Main observation
No dropout, 10 epochs	41.30%	Training was 99.10%: severe overfitting
Dropout, 10 epochs	48.80%	Better generalization

Version	Final validation accuracy	Main observation
Dropout + augmentation, 20 epochs	53.60%	Also trained longer
Larger model, 20 epochs	54.40%	Capacity and dropout both changed

- **Why convolution for images?** It uses neighboring pixels and shares the same filter across positions. A fully connected layer does not build in this local structure or weight sharing.
- **Filters versus kernel size?** More filters learn more feature maps. A larger kernel covers a larger local area. Your larger model increased filter counts, but kept 3x3 kernels.
- **What does pooling do?** Reduces spatial resolution, computation and sensitivity to small shifts.
- **Dropout versus augmentation?** Dropout randomly zeros activations during training; augmentation changes training images while preserving their labels. Both discourage memorization. Standard validation disables these random changes.

Related slides: 4.1, pp. 10-11, 32; 3.4, pp. 32-34.

Lab 3 - `2_SimpleCNN_40epoch.ipynb`: longer experiment

The larger CNN finished at **57.40% validation accuracy**, with a peak of **61.30% at epoch 38**. This is a separate run: do not mix its numbers with the previous notebook.

Does more training always help? No: the final epoch was worse than the best one. Validation-based checkpointing can retain a better model. Capacity, dropout and training duration changed together, so the result does not isolate one factor.

Lab 3 - `3_CNN_Finetuning.ipynb`: transfer learning

What you did: loaded ImageNet-pretrained MobileNetV2, used its matching preprocessing, and replaced the classifier with a five-class head. First trained that head with the backbone frozen, then tested fine-tuning strategies.

Strategy	Validation accuracy
Frozen backbone; train new head	90.90% final
All layers, learning rate 0.0001	71.70% final
All layers, learning rate 0.00001	88.30% at selected checkpoint
Last 20 layers eligible, BatchNorm frozen, rate 0.00001	93.10% at selected checkpoint

- **Feature extraction versus fine-tuning?** Feature extraction keeps pretrained features fixed; fine-tuning updates some or all of those weights.
- **Why did transfer help?** The model already had useful image features. The food dataset did not need to teach everything from random initialization.
- **Why not unfreeze everything?** It may overfit or disrupt useful features. A smaller learning rate and fewer trainable layers can preserve them better.
- **Why freeze BatchNorm?** Preserve useful pretrained normalization behavior. `training=False` uses stored statistics; it does not itself freeze every backbone weight. Variation 2 additionally freezes BatchNorm parameters.
- **Was the comparison controlled?** Both variations restarted from the same frozen-head weights. Checkpoints used minimum validation loss. Do not directly compare these accuracies with the CIFAR-10 network: the datasets differ.

Related slides: 4.1, pp. 57, 67-73.

Lab 5 - `ORIGINAL_FULL.ipynb`: next-character prediction

What you did: trained a decoder-only Transformer on Tiny Shakespeare, using 65 character tokens and a 90%/10% train/validation split. Each target is the next character: the input sequence shifted by one position.

Result: the bigram baseline reached validation loss **2.484**, versus **1.569** for full GPT. Perplexity fell from about **12.0 to 4.8**. GPT's best recorded validation loss was **1.4885 at step 2,500**; later validation loss rose while training loss fell, showing overfitting.

- **Bigram versus GPT?** Bigram uses only the current character. GPT combines information from the preceding context.
- **Queries, keys and values?** A query describes what a position looks for; keys determine how well other positions match; values supply the information combined using attention weights. All are learned projections.
- **Why a causal mask?** It prevents attention to future characters, which would reveal training targets. Training predicts all positions in parallel; generation samples one character, appends it, and repeats.
- **What else is in a block?** Several attention heads mix information across positions; a feed-forward network processes each position. Residual connections help information and gradients pass through layers; layer normalization stabilizes activations. Your implementation normalizes before attention and the feed-forward network.
- **What is perplexity?** Here it is `exp(cross-entropy)`: lower means better next-character predictions. It is not an accuracy percentage.
- **Temperature and top-k?** Low temperature concentrates probabilities; high temperature adds variety and often mistakes. Top-k allows only the k most likely characters. These change sampling, not the learned weights.

Related slides: 5.2, pp. 9-13, 23-27, 30-31.

Lab 5 - `ORIGINAL_FAST.ipynb`: reference for the experiments

Four layers, four heads per layer, 128 embedding dimensions, 128-character context and 3,000 updates. Final validation loss: **1.5966**. Compare each modification with this model, using the same fast settings and seed, rather than with the differently configured full model.

Lab 5 - `ABLATION_SCALING_FAST.ipynb`

Removed `1/sqrt(head_size)` from the attention scores. Validation loss was **1.5918**: essentially unchanged. Some heads showed sharper attention spots.

Why scale attention? Dot products tend to grow with head dimension; scaling helps prevent softmax becoming overly concentrated and gradients becoming small. A small effect in this run does not prove scaling is unnecessary in general. *Slides:* 5.2, p. 13.

Lab 5 - `ABLATION_NO_POSITION_FAST.ipynb`

Removed learned **position** embeddings, not token embeddings. Loss rose to **1.7943**, and generated words became more broken. First-layer attention was more spread out; diagonal patterns appeared later.

Why did this hurt? Token embeddings identify characters; position information helps represent their order and distance. The causal mask still provides some ordering information, so the model is not completely order-blind. *Slides:* 5.2, pp. 15-17.

Lab 5 - `ROPE_FAST.ipynb`

Replaced additive learned positions with position-dependent rotations of query/key pairs in every head. Values were unchanged. Loss improved to **1.5445**, and last-layer attention stayed more focused near the diagonal.

Why RoPE? Rotated query/key dot products depend on relative position as well as content. It removes the learned position table, but adds rotation operations; fewer parameters do not guarantee faster training. Our context remained 128 characters, so we did not test longer-context performance. *Slides:* 5.2, p. 17.

Reading an attention plot correctly

Rows are querying positions; columns are attended-to positions. A bright diagonal means attention to the current or nearby characters; a vertical band means several positions attend to the same earlier character. The upper triangle is masked. Heads are parallel attention mechanisms within a layer; layers are stacked. These plots describe attention, but do not prove understanding or fully explain a prediction.

Slide references are PDF page numbers in the corresponding files under `Lecture Slides/`. Focus on explaining the results and mechanisms, rather than memorizing all settings.